Empresa Ejemplo S.A.
Gerencia de Arquitectura y Desarrollo de Software

Estándar de Diseño y Desarrollo de APIs

APIs REST con ASP.NET Core

Código	Versión	Fecha de aprobación	Estado
EST-API-001	2.0	2026-01-19	Vigente

Aprobado por: Comité de Arquitectura

Documento de prueba con contenido ficticio. La empresa y las reglas son un ejemplo para pruebas de un asistente RAG; no corresponden a ninguna organización real.

Control de cambios

Versión	Fecha	Descripción del cambio
1.0	2021-10-04	Versión inicial.
1.1	2023-06-12	Problem Details para errores y versionado en la URL.
2.0	2026-01-19	Alineación con OWASP API Security Top 10 2023. Paginación obligatoria, idempotencia y catálogo de APIs.

Contenido

Control de cambios . . . 2
1. Objetivo . . . 3
2. Alcance . . . 3
3. Definiciones y niveles de cumplimiento . . . 3
4. Diseño de recursos y URLs . . . 3
5. Versionado . . . 4
6. Respuestas y manejo de errores . . . 5
7. Seguridad de APIs . . . 6
8. Implementación en ASP.NET Core . . . 7
9. Documentación y gobierno . . . 8
10. Relación con otros estándares . . . 8

1. Objetivo

Asegurar que las APIs de la empresa sean consistentes, seguras, documentadas y fáciles de consumir, de modo que un consumidor que conoce una API pueda usar cualquier otra sin sorpresas.

2. Alcance

Aplica a todas las APIs HTTP expuestas por sistemas de la empresa, internas o externas, construidas con ASP.NET Core. Las APIs que consumen aplicaciones Angular, móviles y de escritorio están incluidas. Complementa a EST-SEG-001 (seguridad) y EST-WEB-001 (convenciones de código C#).

3. Definiciones y niveles de cumplimiento

Término	Definición
Obligatorio	La regla debe cumplirse en todo desarrollo nuevo y en todo cambio sobre código existente. Su incumplimiento bloquea el pase a producción salvo excepción aprobada.
Recomendado	Buena práctica esperada. Su incumplimiento no bloquea el pase, pero debe justificarse en la revisión de código.
Prohibido	Práctica que no debe aparecer en el código. Equivale a una regla obligatoria en sentido negativo.
Excepción	Autorización temporal (máximo 6 meses) para no cumplir una regla obligatoria. La aprueba el Comité de Arquitectura y, si la regla es de seguridad, también Seguridad de la Información. Se registra en el formulario FOR-ARQ-07.
Recurso	Entidad de negocio expuesta por la API e identificada por una URL (por ejemplo un cliente).
Idempotente	Operación que produce el mismo resultado si se ejecuta una o varias veces.
Problem Details	Formato estándar de errores HTTP en JSON definido por el RFC 9457.
BFF	Backend for Frontend: API dedicada a un único frontend.

4. Diseño de recursos y URLs

4.1 API-DIS-001 · Nombres de recursos en las URLs

Nivel: Obligatorio | **Aplica a:** API

Las URLs usan sustantivos en plural, en minúsculas y kebab-case, con el prefijo `/api/v{n}`: `/api/v1/clientes/{clienteId}/cuentas`. Está prohibido usar verbos en la URL (`/obtenerClientes`, `/clientes/eliminar/5`), extensiones (`.json`) y mayúsculas. Las acciones que no encajan en CRUD se modelan como subrecursos (`POST /api/v1/pedidos/{pedidoId}/anulacion`).

Ejemplo incorrecto

```
GET  /api/ObtenerClientes
POST /api/clientes/eliminar/5
GET  /api/v1/Cliente/5/CuentasBancarias
```

Ejemplo correcto

```
GET    /api/v1/clientes
DELETE /api/v1/clientes/5
GET    /api/v1/clientes/5/cuentas-bancarias
```

4.2 API-DIS-002 · Uso de los métodos HTTP

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** ASVS V13.2.1

GET consulta y no modifica estado; POST crea un recurso o ejecuta una acción; PUT reemplaza el recurso completo; PATCH modifica parcialmente (JSON Merge Patch); DELETE elimina. GET, PUT y DELETE deben ser idempotentes. Los métodos no usados por un recurso responden 405.

4.3 API-DIS-003 · Profundidad de anidamiento

Nivel: Obligatorio | **Aplica a:** API

Las URLs tienen como máximo dos niveles de anidamiento (`/clientes/{id}/cuentas`). Para relaciones más profundas se expone el recurso en su propia ruta con filtros (`/movimientos?cuentaId=10`).

4.4 API-DIS-004 · Formato de los datos

Nivel: Obligatorio | **Aplica a:** API

JSON en UTF-8 con propiedades en camelCase. Fechas en ISO 8601 con zona horaria UTC (`2026-03-10T15:04:05Z`). Montos como número decimal acompañados de la moneda en código ISO 4217 (`{"monto": 1500.50, "moneda": "PEN"}`). Enumeraciones como texto, no como número. Los nulos se omiten de la respuesta.

Ejemplo incorrecto

```
{ "Cliente_ID": 15, "FechaRegistro": "10/03/2026", "Monto": "1,500.50", "Estado": 2 }
```

Ejemplo correcto

```
{ "clienteId": 15, "fechaRegistro": "2026-03-10T15:04:05Z",
  "monto": 1500.50, "moneda": "PEN", "estado": "Activo" }
```

4.5 API-DIS-005 · Contrato primero con OpenAPI

Nivel: Obligatorio | **Aplica a:** API

Toda API publica su contrato OpenAPI 3 versionado en el repositorio y en el catálogo de APIs, con descripción y ejemplo de cada operación, esquemas de request y response, y códigos de error posibles. El contrato se revisa antes de iniciar la implementación.

5. Versionado

5.1 API-VER-001 · Versión en la URL

Nivel: Obligatorio | **Aplica a:** API

La versión mayor forma parte de la URL (`/api/v1/`). Agregar campos opcionales o nuevas operaciones no cambia la versión. Eliminar o renombrar campos, cambiar tipos o hacer obligatorio un campo opcional son cambios incompatibles y requieren una nueva versión mayor.

5.2 API-VER-002 · Deprecación de versiones

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** OWASP API9:2023

Una versión deprecada responde con las cabeceras `Deprecation` y `Sunset` y convive como mínimo 6 meses con la nueva versión. Al cumplirse la fecha de retiro, la versión se elimina del gateway y del catálogo; no deben quedar versiones antiguas expuestas sin dueño.

6. Respuestas y manejo de errores

6.1 API-RES-001 · Códigos de estado HTTP

Nivel: Obligatorio | **Aplica a:** API

200 consulta o modificación exitosa; 201 recurso creado con cabecera `Location`; 204 éxito sin contenido; 400 petición mal formada o validación fallida; 401 sin autenticación; 403 autenticado sin permiso; 404 recurso inexistente o no visible para el usuario; 409 conflicto de estado; 422 regla de negocio incumplida; 429 límite de tasa superado; 500 error no controlado. Está prohibido responder 200 con un error en el cuerpo.

Ejemplo incorrecto

```
return Ok(new { exito = false, mensaje = "Cliente no encontrado" });
```

Ejemplo correcto

```
if (cliente is null) return NotFound();
var creado = await _clientes.CrearAsync(dto, ct);
return CreatedAtAction(nameof(Obtener), new { clienteId = creado.ClienteId }, creado);
```

6.2 API-RES-002 · Errores con Problem Details

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** EST-SEG-001 SEG-LOG-003

Toda respuesta de error usa Problem Details (RFC 9457, `application/problem+json`) con `type`, `title`, `status`, `detail` y la extensión `traceId`. Los errores de validación incluyen `errors` por campo. Está prohibido incluir trazas de pila, sentencias SQL o mensajes internos de excepción (ver SEG-LOG-003). En ASP.NET Core se habilita con `AddProblemDetails()`.

Ejemplo correcto

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json

{ "type": "https://api.empresa-ejemplo.pe/errores/limite-credito",
  "title": "Límite de crédito excedido",
  "status": 422,
  "detail": "El monto solicitado supera el límite disponible del cliente.",
  "traceId": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01" }
```

6.3 API-RES-003 · Paginación de colecciones

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** OWASP API4:2023

Toda operación que devuelve una colección es paginada. Se usan los parámetros `page` (desde 1) y `pageSize` (por defecto 20, máximo 100) y la respuesta incluye `items`, `page`, `pageSize` y `totalItems`. Para colecciones muy grandes o en tiempo real se usa paginación por cursor. Está prohibido devolver colecciones completas sin límite.

Ejemplo correcto

```
GET /api/v1/clientes?page=2&pageSize=20&estado=Activo

{ "items": [ ... ], "page": 2, "pageSize": 20, "totalItems": 347 }
```

6.4 API-RES-004 · DTOs en lugar de entidades

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** OWASP API3:2023

Las operaciones reciben y devuelven DTOs específicos (`CrearClienteRequest`, `ClienteResponse`). Está prohibido exponer entidades de EF Core o de dominio directamente, porque se filtran campos internos y se permite la asignación masiva de propiedades que el usuario no debe modificar.

Ejemplo incorrecto

```
[HttpPut("{clienteId}")]
public async Task<IActionResult> Actualizar(int clienteId, Cliente cliente)
{   // el usuario puede enviar "limiteCredito" o "esVip" y se guardan
    _contexto.Update(cliente); await _contexto.SaveChangesAsync(); return NoContent();
}
```

Ejemplo correcto

```
public record ActualizarClienteRequest(string Nombre, string Telefono, string Correo);

[HttpPut("{clienteId}")]
public async Task<IActionResult> Actualizar(
    int clienteId, ActualizarClienteRequest req, CancellationToken ct)
{
    await _clientes.ActualizarContactoAsync(clienteId, req, ct);
    return NoContent();
}
```

7. Seguridad de APIs

7.1 API-SEG-001 · Autenticación de APIs

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** ASVS V13.1.3; OWASP API2:2023; EST-SEG-001 SEG-AUT-006

Las APIs se protegen con OAuth 2.0 / OpenID Connect sobre Microsoft Entra ID y validan JWT según SEG-AUT-006. Las claves de API solo se permiten para integraciones servidor a servidor con terceros, se envían en una cabecera (`X-Api-Key`) y nunca en la URL, y se gestionan en API Management. Todo endpoint requiere autenticación salvo los de salud y los marcados explícitamente como públicos.

7.2 API-SEG-002 · Autorización por operación y por objeto

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** OWASP API1:2023; OWASP API5:2023; EST-SEG-001 SEG-ACC-002

Cada operación declara la política de autorización que exige (`[Authorize(Policy = "...")]`) con base en scopes o roles; la autorización global por defecto exige usuario autenticado. Además, las operaciones sobre un recurso concreto verifican la propiedad del objeto (ver SEG-ACC-002).

Ejemplo incorrecto

```
[HttpDelete("{usuarioId}")]
public async Task<IActionResult> Eliminar(int usuarioId) { ... }   // cualquier usuario autenticado
```

Ejemplo correcto

```
[HttpDelete("{usuarioId}")]
[Authorize(Policy = Politicas.AdministrarUsuarios)]
public async Task<IActionResult> Eliminar(int usuarioId, CancellationToken ct) { ... }
```

7.3 API-SEG-003 · Límites de consumo

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** OWASP API4:2023; OWASP API6:2023

Toda API aplica limitación de tasa por cliente (`AddRateLimiter` en ASP.NET Core o políticas de API Management), un tamaño máximo de petición de 1 MB (salvo operaciones de carga de archivos), tiempos máximos de ejecución y responde 429 con la cabecera `Retry-After` cuando se supera el límite.

Ejemplo correcto

```
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    o.AddFixedWindowLimiter("porCliente", l =>
    {
        l.PermitLimit = 100;
        l.Window = TimeSpan.FromMinutes(1);
    });
});
```

7.4 API-SEG-004 · Validación del contrato de entrada

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** ASVS V13.2.2; EST-SEG-001 SEG-INY-002

Todo request se valida antes de procesarse (FluentValidation o DataAnnotations): tipos, campos obligatorios, longitudes, rangos y formatos. La deserialización rechaza propiedades desconocidas (`JsonUnmappedMemberHandling.Disallow`). Una validación fallida responde 400 con Problem Details.

7.5 API-SEG-005 · Prevención de SSRF

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** OWASP API7:2023; OWASP A10:2021

Si la API debe obtener un recurso a partir de una URL recibida, valida el destino contra una lista blanca de dominios y esquemas (solo `https`), rechaza direcciones IP privadas, de loopback y de metadatos de la nube, y no sigue redirecciones automáticamente.

7.6 API-SEG-006 · Consumo seguro de APIs de terceros

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** OWASP API10:2023

Las respuestas de APIs externas se tratan como entrada no confiable y se validan. Toda llamada saliente tiene tiempo de espera, reintentos con espera exponencial e interruptor de circuito (Polly / `Microsoft.Extensions.Http.Resilience`), y usa TLS con validación de certificado.

7.7 API-SEG-007 · Datos sensibles fuera de la URL

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** ASVS V8.3.1

Está prohibido enviar tokens, contraseñas, números de documento o de tarjeta en la URL o en la query string, porque quedan en logs de servidores, proxies y en el historial del navegador. Se envían en el cuerpo o en cabeceras.

Ejemplo incorrecto

```
GET /api/v1/clientes/buscar?dni=45678912&token=eyJhbGciOi...
```

Ejemplo correcto

```
POST /api/v1/clientes/busqueda
Authorization: Bearer eyJhbGciOi...
{ "numeroDocumento": "45678912" }
```

8. Implementación en ASP.NET Core

8.1 API-NET-001 · Controladores delgados

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** EST-WEB-001 WEB-ARQ-001

Los controladores usan `[ApiController]`, rutas por atributo y devuelven `ActionResult<T>` o `IResult`. Solo orquestan: reciben el request, llaman a un caso de uso de la capa Application y traducen el resultado a HTTP. Está prohibido acceder al `DbContext` o contener reglas de negocio en el controlador.

8.2 API-NET-002 · Uso de HttpClient

Nivel: Obligatorio | **Aplica a:** API

Los clientes HTTP se registran con `IHttpClientFactory` como clientes tipados. Está prohibido crear `new HttpClient()` por petición (agota los sockets) y configurar `HttpClientHandler` para aceptar cualquier certificado.

Ejemplo incorrecto

```
using var http = new HttpClient();
var r = await http.GetAsync("https://servicio-externo/tipo-cambio");
```

Ejemplo correcto

```
builder.Services.AddHttpClient<ITipoCambioClient, TipoCambioClient>(c =>
    c.BaseAddress = new Uri(builder.Configuration["Servicios:TipoCambio"]!))
    .AddStandardResilienceHandler();
```

8.3 API-NET-003 · Health checks

Nivel: Obligatorio | **Aplica a:** API

Toda API expone `/health/live` (el proceso responde) y `/health/ready` (dependencias como base de datos disponibles). Las respuestas no incluyen cadenas de conexión, versiones ni detalles internos cuando se consultan desde fuera de la red de monitoreo.

8.4 API-NET-004 · Correlación y trazabilidad

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** EST-SEG-001 SEG-LOG-002

La API acepta y propaga la cabecera `X-Correlation-Id` (la genera si no existe), la incluye en todos los logs y en las respuestas de error, y se instrumenta con OpenTelemetry para trazas y métricas.

8.5 API-NET-005 · Idempotencia de operaciones críticas

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** OWASP API6:2023

Las operaciones POST que generan efectos monetarios o no reversibles (pagos, transferencias, emisión de documentos) exigen la cabecera `Idempotency-Key`. Si se recibe una clave ya procesada en las últimas 24 horas, se devuelve la respuesta original sin volver a ejecutar la operación.

9. Documentación y gobierno

9.1 API-GOB-001 · Registro en el catálogo de APIs

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** OWASP API9:2023

Toda API se registra en Azure API Management con: dueño funcional y técnico, versión, ambientes, clasificación de los datos que expone, consumidores autorizados y fecha de retiro si está deprecada.

9.2 API-GOB-002 · Documentación interactiva

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** OWASP API8:2023

Swagger UI / Scalar se habilitan solo en los ambientes de desarrollo y certificación. En producción el contrato OpenAPI se consulta en el portal de desarrolladores de API Management.

Ejemplo correcto

```
if (app.Environment.IsDevelopment() || app.Environment.IsStaging())
{
    app.MapOpenApi();
    app.UseSwaggerUI(o => o.SwaggerEndpoint("/openapi/v1.json", "v1"));
}
```

10. Relación con otros estándares

Las reglas de seguridad de EST-SEG-001 aplican a todas las APIs aunque no se repitan aquí. Las convenciones de código C# y de base de datos están en EST-WEB-001.